EEE3097/8/9S Micromouse 2026: Milestone 1 Instructions

Closed-Loop Path Traversal (The Square Run)

---

1. Objective

Design and implement a closed-loop controller that guides your Micromouse robot to traverse a perfect **1.0m x 1.0m square** on the floor and stop autonomously. Your design must use closed-loop feedback (such as gyroscope yaw heading alignment and quadrature encoder distances) to remain robust against traction loss, slip, and motor asymmetries.

Specific Learning Objectives:

- Configure and read quadrature wheel encoders to track linear distance.
- Interface with the MPU6050 IMU gyroscope and integrate angular rate to track heading.
- Formulate a Proportional (P) or PID controller to synchronize wheel velocities and steer the mouse.
- Implement a finite state machine (FSM) to transition between straight trajectories and spin-in-place turns.
- Design safety guards to handle motor saturation and battery voltage sag.

---

2. Step-by-Step Implementation Guide

Step 1: Sensor Verification & Calibration

- Place the mouse on a flat surface. Write a test script to read raw encoder ticks and gyro yaw rate.
- Verify that pushing the mouse forward increases both encoder counts symmetrically.
- Rotate the mouse 360° manually. Verify that your gyro integration math returns exactly 360° (or 2π radians). Adjust your gyroscope scale factor parameters if you see scaling errors.

Step 2: Dual-Motor Speed Synchronization

- Implement a controller that adjusts left and right motor PWM signals dynamically so both wheels rotate at the same linear velocity.
- Test this by running the mouse straight on the floor. If it veers to one side, tune your feedback gains to minimize the drift.

Step 3: Closed-Loop Heading Correction (Steering)

- Add your integrated gyroscope yaw angle to your straight-run feedback loop.
- Set your target yaw heading to 0°. If the mouse drifts or is physically bumped, your controller must adjust the wheel speeds differentially (steering) to return to 0°.

Step 4: Spin-in-Place Turning

- Implement a precise 90° right turn state.
- The mouse should spin in place (left wheel forward, right wheel reverse) until the integrated gyroscope yaw angle matches exactly +90° relative to the heading before the turn.
- Ensure your turn controller includes a settling window to prevent overshoot and oscillations.

Step 5: Finite State Machine Integration

- Combine these states into a sequential state machine:

$$\text{DRIVE\_FORWARD\_1 (1.0m)} \rightarrow \text{TURN\_RIGHT\_1 (90}^{\circ}\text{)} \rightarrow \cdots \rightarrow \text{STOP}$$

- The mouse must come to a complete, autonomous stop at the end of the 4th turn.

---

3. Deliverables (Gradescope Submission)

To make submission simple and prevent errors, a packaging tool is provided. Run the following command from your repository root:

```
python tools/package_submission.py --task milestone1 --src workspace/task1_square/
```

This script will perform dynamic diagnostic checks: * **Local Syntax Guard:** The script runs a local syntax compile check. If your Python code has indentation or formatting errors, the packager will abort and pinpoint the error line so you can fix it before zipping. * **Auto-Generated Package:** It automatically packages all code, subdirectories, models, and telemetry logs into a single `submission_milestone1.zip` in your project root.

Upload this ZIP file directly to Gradescope. The autograder will immediately compile and run your submission in co-simulation, printing a detailed test result feedback log (detailing syntax, compilation, runtime errors, or file mismatches) directly to your Gradescope portal in real-time.

The ZIP package will automatically contain: 1. **Your Controller Code:** * **Python track:** All `.py` scripts and subfolders recursively from your workspace. * **Simulink track:** Your model file (`.slx`) and the generated C code-generation directory (`*_ert_rtw/`). 2. **Physical Telemetry Log (`run_log.jsonl`):** * The raw telemetry log file extracted from your physical mouse using `python tools/dump_logs.py`. 3. **Physical Run Video (`run_video.mp4`):** * A single, unedited video demonstrating the physical run. (Note: Video can also be uploaded separately on Gradescope depending on the portal settings, but the packager includes it if present).

---

4. Video Requirements & Academic Honesty Declaration

To verify that your physical run is authentic and represents your own work, the video must strictly adhere to the following sequence: 1. **Student Card Close-up:** The video **MUST start with a clear, readable close-up of your physical Student Card** for at least 3 seconds. In doing so, you declare that the submission represents your own work. 2. **Setup:** Pan the camera to show the mouse positioned at the starting corner of the 1.0m x 1.0m grid. 3. **Traversal:** Capture the complete run without cuts. The mouse must drive 1.0m straight, turn 90° right, and repeat this 4 times to form a square, coming to an autonomous stop.

---

5. Code & Log Correlation Verification (Anti-Cheat Check)

The autograder uses two fields in your log's header line to verify authenticity:

```
{"log_header":1,"uid":"066AFF514885864967083830","hash":4017325881}
```

- **Hardware ID Check:** The `"uid"` field represents your microcontroller's unique device ID. While this is not registered in advance, the course convenors check the submitted logs for duplicate UIDs. Submitting logs with identical UIDs under different student accounts indicates shared files/hardware runs and will trigger a plagiarism audit.

- **Code Match Check:** The `"hash"` field is a 32-bit FNV-1a checksum computed in hardware based on the code loaded into the mouse. The autograder will compile your submitted script/binary locally and verify that the resulting checksum matches the hash inside the log. **Mismatched hashes will result in an immediate submission rejection.**

---

6. Evaluation Criteria

Your submission will be graded on two parts:

A. Visual & Log Audit (Manual/Physical Check)

- **Square Accuracy:** The mouse must stay within a ± 10 cm margin of the 1m square boundary.
- **Autonomous Stop:** The mouse must come to a complete halt autonomously at the end of the 4th leg.
- **Log Check:** Telemetry variables must show active adjustments (motor speed matching and heading corrections).

B. Co-Simulation Stress Test (Automated Check) The autograder will execute your controller code in the visual simulation testbed under two perturbations: 1. **Motor Asymmetry ($\pm 10\%$ gain offset):** Simulates one motor being weaker than the other. 2. **Wheel Slip & Traction Loss:** Simulates low friction on the floor. *To pass the simulation test, your controller must dynamically adjust to these disturbances to complete the square trajectory.*